Cálculo de Área con Diseño Modular: Enfoque Descendente y Reutilización de Funciones en C++

Aramis Demaria

E.E.S.T. N.º 1 “Eduardo Ader” – Vicente López

Laboratorio de Programación (LPR) – 5.º Año 3.º A-B

Prof. Mansilla Muñoz York Elías

2026

Resumen

Se implementa una calculadora de área del círculo bajo metodología descendente. El problema se descompone en el módulo *calcularAreaCirculo*, declarado mediante prototipo antes de *main* y definido después. La constante PI se protege con el calificador *const* para garantizar inmutabilidad en la memoria. Un filtro de consistencia rechaza radios no positivos. El programa muestra el área y representa una diana ASCII escalada. Las pruebas con radios de 3, 7.5 y 15 cm validan el cálculo y la clasificación MINI, ESTÁNDAR y GIGANTE. El diseño modular facilita la depuración y la reutilización, competencias clave para el Proyecto Integrador Anual.

Palabras clave: diseño descendente, modularización, prototipo de función, const, área del círculo, diana ASCII

Abstract

A circle area calculator is implemented under a top-down methodology. The problem is decomposed into the *calcularAreaCirculo* module, declared via a prototype before *main* and defined after it. The PI constant is protected with *const* to ensure memory immutability. A consistency filter rejects non-positive radii. The program displays the area and renders a scaled ASCII target. Tests with radii of 3, 7.5, and 15 cm validate the calculation and the MINI, STANDARD, and GIANT classification. Modular design facilitates debugging and reuse, key competencies for the Annual Integrative Project.

Keywords: top-down design, modularization, function prototype, const, circle area, ASCII target

Introducción

El diseño descendente consiste en abordar un problema complejo dividiéndolo en módulos especializados. En C++ esto se concreta con funciones diseñadas por el programador: cada función resuelve una subtarea y la función principal coordina su

ejecución.

El caso de estudio es el cálculo del área definida como $A = \pi r^2$. Aunque la fórmula es trivial, el énfasis radica en la arquitectura: prototipo, invocación y definición separados, y protección de invariantes (PI).

Revisión de la Literatura

El diseño descendente va de lo general a lo particular, subdividiendo el sistema en módulos independientes. Modularizar consiste en dividir el programa en funciones para mejorar legibilidad, mantenimiento y localización de errores. El prototipo es la declaración anticipada que informa al compilador sobre nombre, valor de retorno y parámetros. El calificador *const* bloquea la variable en la memoria e impide su modificación; todo intento de reasignación produce error de compilación (Microsoft, 2024; Stroustrup, 2013).

Fase	Qué hace	Ubicación en el código
Prototipo	Anuncia la función	Antes de main: double calcularAreaCirculo(double);
Invocación	Utiliza el módulo	En main: double areaFinal = calcularAreaCirculo(radio);
Definición	Implementa la fórmula	Después de main: { const PI; return PI*r*r; }

Nota. Ciclo de vida de una función modular en C++.

Método

Se organizó la carpeta */calculoarea* con *src/main.cpp*, *docs/* y *capturas/*. La función principal solicita *radioEstudiante*, aplica el filtro *if (radio <= 0) return 1* para terminación controlada, invoca el módulo y muestra *areaFinal*. La diana ASCII se implementa con estructuras *if / else if* según el rango del radio (≤ 5, ≤ 12 o > 12).

La función *calcularAreaCirculo* declara *const double PI = 3.1415926535* y retorna *PI * r * r*. La compilación se ejecutó con *g++ src/main.cpp -o src/area.exe* y la ejecución con *.\src\area.exe*. El diagrama de flujo se conserva en *capturas/diagrama_flujo.png*.

Resultados

Con radio de 7.5 cm el área resultó 176.715 cm², coincidente con el cálculo manual. Las clasificaciones fueron: 3 cm, MINI; 7.5 cm, ESTÁNDAR; y 15 cm, GIGANTE. El filtro rechazó los valores 0 y −5 con mensaje de error y código de salida 1.

Figura 1. Terminal compilando y ejecutando con radio 7.5 (capturas/compilacion_area.png). Figura 2. Diagrama de flujo modular (capturas/diagrama_flujo.png).

Discusión

Sin prototipo, el compilador realiza una sola pasada de arriba hacia abajo y, al encontrar la llamada *calcularAreaCirculo(radio)* dentro de *main* sin declaración previa, genera el error de identificador no declarado. El prototipo constituye el contrato anticipado.

Admitir radio menor o igual a cero carece de sentido geométrico y generaría áreas nulas o negativas, además de invalidar la representación de la diana. Declarar PI como *const* es esencial en sistemas críticos —por ejemplo, aviónica o instrumental médico—, donde una mutación accidental alteraría todos los cálculos dependientes y el compilador actúa como mecanismo de protección de integridad.

Conclusión

El enfoque modular se aplicó correctamente al completar las líneas faltantes de invocación y salida y la definición con PI constante. El programa es mantenible, verificable y reutilizable. Como extensión futura se propone extraer el módulo a un encabezado *geometria.h* para su reutilización en el Proyecto Integrador Anual.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

cplusplus.com. (2024). *Functions in C++*. https://cplusplus.com/doc/tutorial/functions/

Microsoft. (2024). *const (C++)*. https://learn.microsoft.com/es-es/cpp/cpp/const-cpp

Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.